

ELO329 - Diseño y Programación Orientados a Objetos

Desarrollo en Qt 6

Agustín J. González
Patricio Olivares R.

Universidad Técnica Federico Santa María



Objetivo y ruta

Qt permite construir interfaces gráficas C++ sin programar directamente contra cada sistema operativo.

En esta clase conectamos Qt con temas de ELO329:

- clases, herencia, composición y encapsulamiento.
- eventos, signals y slots.
- interfaz gráfica separada de la lógica.
- dibujo, escenas y animaciones.

La meta no es memorizar widgets. La meta es estructurar una aplicación Qt mantenible.

Qt 6 en este curso

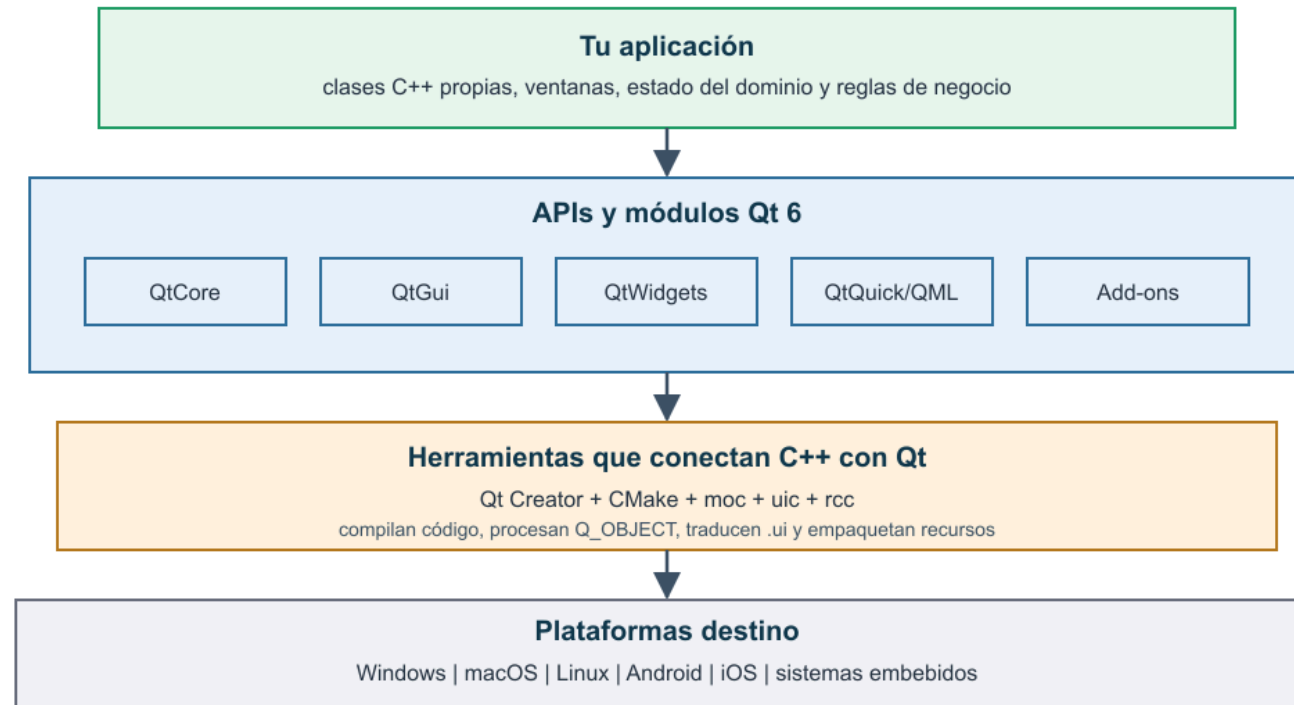
Flujo de trabajo:

- `CMakeLists.txt` para configurar.
- `QApplication` para iniciar el ciclo de eventos.
- Qt Widgets y Qt Designer para la interfaz.
- signals y slots con sintaxis tipada.

`qmake` sigue existiendo. Para trabajos nuevos del curso, preferimos CMake.

Arquitectura general

Qt 6: una capa de abstracción para aplicaciones C++



La aplicación se escribe contra APIs Qt, y Qt abstrae diferencias entre Windows, macOS, Linux y otras plataformas.

Módulos y enfoques de interfaz

Componente	Rol
QtCore	QObject , eventos, archivos, hilos, temporizadores
QtGui	dibujo, fuentes, imágenes, entrada de usuario
QtWidgets	controles de escritorio como QWidget , QMainWindow , QPushButton
QtQuick/QML	interfaces declarativas, fluidas y animadas

En ELO329 partimos con Qt Widgets porque trabaja directamente con C++ y refuerza POO.

Herramientas y proyecto base

Qt Creator integra edición, diseño visual, compilación, ejecución y depuración.

```
MiApp/  
├─ CMakeLists.txt  
├─ main.cpp  
├─ mainwindow.h  
├─ mainwindow.cpp  
├─ mainwindow.ui  
└─ build/
```

`mainwindow.ui` describe la interfaz. `mainwindow.h/.cpp` define el comportamiento.

Qué genera Qt Creator

Cuando se crea un proyecto `Qt Widgets Application`, Qt Creator prepara una estructura mínima.

Archivo	Rol
<code>CMakeLists.txt</code>	declara fuentes, módulos Qt y reglas de compilación
<code>main.cpp</code>	crea <code>QApplication</code> , muestra la ventana y entra al ciclo de eventos
<code>mainwindow.ui</code>	guarda la interfaz diseñada visualmente
<code>mainwindow.h/.cpp</code>	define la clase que controla esa ventana
<code>build/</code>	contiene archivos generados por CMake, no es código fuente

El Kit seleccionado combina compilador, versión de Qt y generador de build.

CMake mínimo

```
cmake_minimum_required(VERSION 3.16)
project>HelloQt LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(Qt6 REQUIRED COMPONENTS Widgets)
qt_standard_project_setup()

qt_add_executable>HelloQt
    main.cpp mainwindow.h mainwindow.cpp mainwindow.ui
)

target_link_libraries>HelloQt PRIVATE Qt6::Widgets)
```

`qt_standard_project_setup()` activa reglas como `AUTOMOC` y `AUTOUIIC` .

main.cpp y ciclo de eventos

```
#include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    MainWindow window;
    window.show();

    return app.exec();
}
```

`app.exec()` mantiene activa la aplicación esperando eventos.

Flujo al ejecutar una app Qt Widgets

1. `QApplication` inicializa Qt y recibe los argumentos del programa.
2. `MainWindow window;` construye la ventana principal.
3. `ui->setupUi(this)` crea los widgets descritos en `mainwindow.ui`.
4. `connect(...)` define qué ocurre cuando el usuario interactúa.
5. `window.show()` muestra la ventana.
6. `app.exec()` entra al ciclo de eventos.

La aplicación no avanza como un programa lineal tradicional: queda esperando eventos de teclado, mouse, temporizadores o señales internas.

QObject y ciclo de vida

QObject es la base de gran parte de Qt: organiza objetos, signals/slots, propiedades, eventos y temporizadores.

```
QPushButton *button = new QPushButton("Cerrar", this);  
QTimer *timer = new QTimer(this);
```

Si un QObject tiene padre, el padre destruye a sus hijos.

Widgets y ventana principal

Un widget puede mostrarse, recibir eventos, contener otros widgets y participar en layouts.

`QMainWindow` entrega menú, barras de herramientas, zona central y barra de estado.

Use ``QMainWindow`` para herramientas completas. Use ``QWidget`` para ventanas o formularios simples.

Separar interfaz y lógica

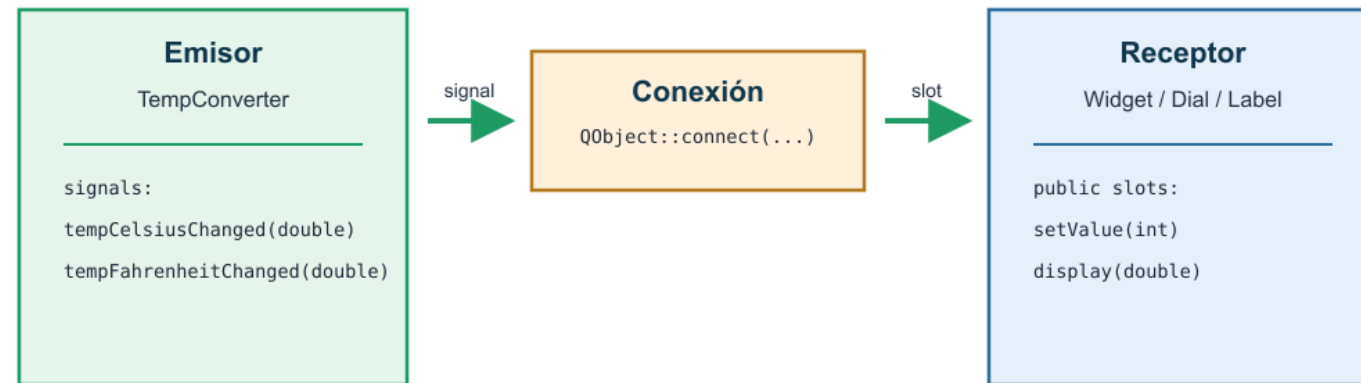
No conviene poner toda la aplicación dentro de `MainWindow`.

Parte	Responsabilidad
<code>MainWindow</code>	leer widgets, actualizar widgets, conectar eventos
Clases de dominio	estado y reglas del problema
Servicios	archivos, red, cálculos, persistencia

La interfaz debe usar el modelo, no convertirse en el modelo.

Signals y slots

Signals y slots: objetos acoplados por contrato, no por dependencia directa



Al emitirse una señal, Qt invoca cada slot conectado con firma compatible.

Propiedad clave para diseño OO

El emisor no necesita saber cuántos receptores existen ni qué harán.

La firma de la señal y del slot define el contrato.

Un signal anuncia que algo ocurrió. Un slot reacciona sin que el emisor conozca directamente a sus receptores.

Declarar y emitir señales

```
class Counter : public QObject {
    Q_OBJECT

public slots:
    void setValue(int value);

signals:
    void valueChanged(int newValue);

private:
    int m_value = 0;
};

void Counter::setValue(int value)
{
    if (value == m_value)
        return;

    m_value = value;
    emit valueChanged(m_value);
}
```

Q_OBJECT y moc

Q_OBJECT permite que moc genere código auxiliar para señales, slots y propiedades.

Ese código no se escribe a mano. CMake lo invoca cuando el archivo está dentro del proyecto.

Si falta...	Síntoma típico
Q_OBJECT	no funcionan señales o slots propios
archivo en CMakeLists.txt	errores de link o undefined reference to vtable
tipo compatible en connect	error de compilación con sintaxis moderna

La ventaja es desacoplar objetos: el emisor anuncia un cambio y no necesita conocer la lógica del receptor.

Conectar objetos

```
Counter a;  
Counter b;  
  
QObject::connect(&a, &Counter::valueChanged,  
                &b, &Counter::setValue);  
  
a.setValue(12);
```

Para widgets:

```
connect(ui->btnSaludar, &QPushButton::clicked,  
        this, &MainWindow::saludar);  
  
void MainWindow::saludar()  
{  
    ui->lblMensaje->setText("Hola desde Qt 6");  
}
```

La sintaxis moderna permite detectar incompatibilidades de tipos en compilación.

qOverload en conexiones

Cuando una señal o slot está sobrecargado, el compilador necesita saber qué firma usar.

```
connect(ui->dial,  
        &QDial::valueChanged,  
        ui->lcdNumber,  
        qOverload<int>(&QLCDNumber::display));
```

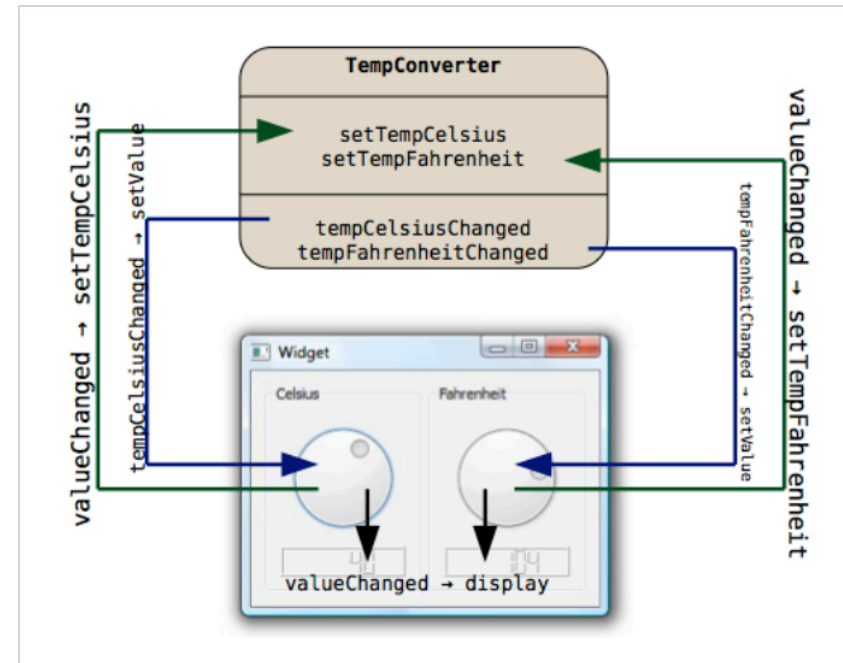
En este ejemplo, `QLCDNumber::display` tiene varias versiones. `qOverload<int>` selecciona la que recibe un `int`.

Use `#include <QOverload>` si el proyecto no lo recibe desde otros encabezados Qt.

Ejemplo: TempConverter

examples/TempConverter muestra signals y slots con una interfaz simple:

- dos controles representan Celsius y Fahrenheit.
- la clase `TempConverter` concentra la conversión.
- los widgets emiten `valueChanged(int)`.
- el modelo emite `tempCelsiusChanged(int)` y `tempFahrenheitChanged(int)`.
- la interfaz se actualiza sin duplicar reglas de conversión en cada widget.



TempConverter: conexiones

El ejemplo conecta widgets con slots del objeto de dominio:

```
connect(ui->celsiusDial, SIGNAL(valueChanged(int)),
        tConverter, SLOT(setTempCelsius(int)));

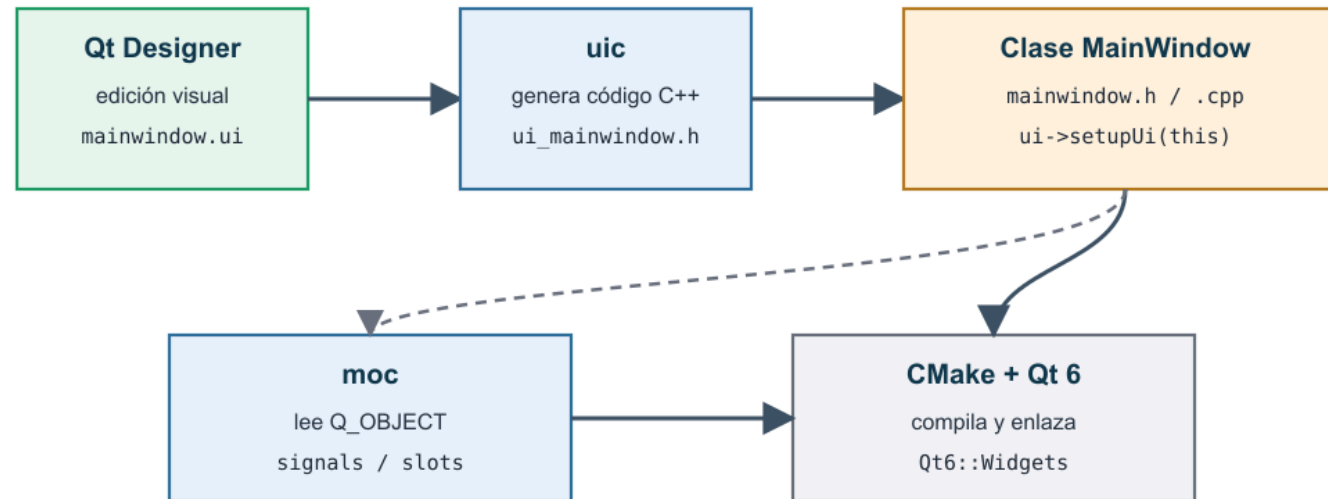
connect(tConverter, SIGNAL(tempFahrenheitChanged(int)),
        ui->fahrenheitDial, SLOT(setValue(int)));
```

La idea importante no es la fórmula de temperatura, sino la comunicación: widgets notifican cambios, una clase calcula, y la interfaz recibe el nuevo estado.

Para código nuevo, prefiera la sintaxis moderna de `connect`. El ejemplo heredado usa macros `SIGNAL` y `SLOT`.

Qt Designer y archivos .ui

De Qt Designer a una ventana ejecutable



El archivo .ui describe apariencia; la clase C++ decide comportamiento.

Designer define apariencia y jerarquía de widgets. C++ define comportamiento.

El archivo `.ui` es XML. Durante la compilación, `uic` genera una clase auxiliar con los widgets declarados en la interfaz.

MainWindow con Designer

```
MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    connect(ui->btnSaludar, &QPushButton::clicked,
            this, &MainWindow::saludar);
}

MainWindow::~MainWindow()
{
    delete ui;
}
```

`setupUi()` crea los widgets definidos en `mainwindow.ui`.

Nombres y layouts

El `objectName` definido para cada Widget agregado permite acceder a este desde C++:

```
ui->lblMensaje->setText("Listo");  
ui->btnGuardar->setEnabled(true);
```

Use nombres claros:

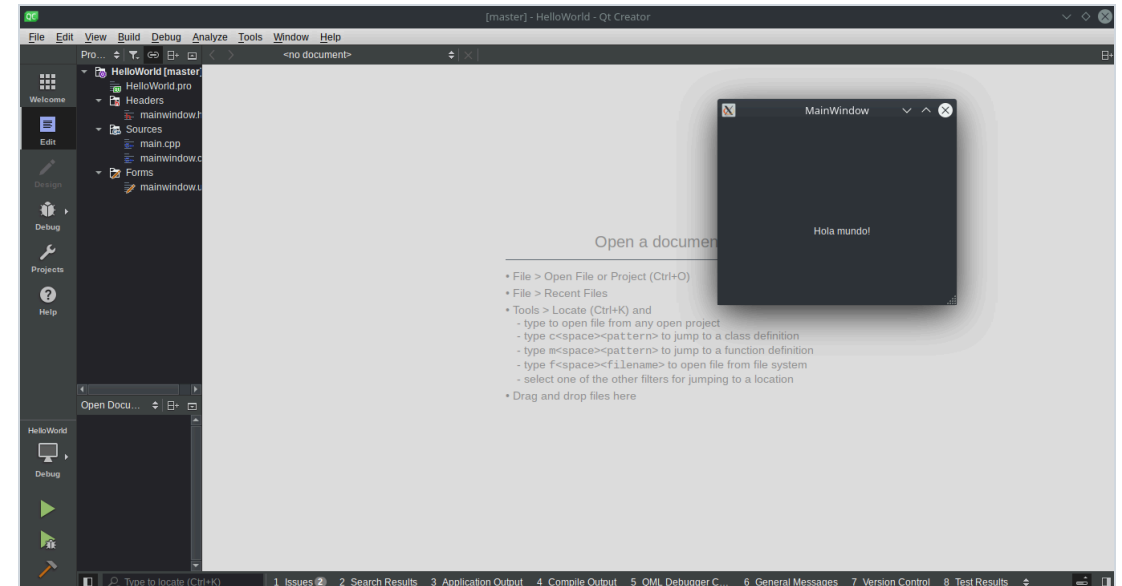
Evitar	Preferir
<code>pushButton</code>	<code>btnGuardar</code>
<code>label_2</code>	<code>lblEstado</code>

Use layouts, no coordenadas fijas, para que la interfaz escale bien.

Espacio de ejemplo: HelloWorld con Qt Widgets

Objetivo de la demo:

1. crear un proyecto Qt Widgets Application .
2. abrir `mainwindow.ui` .
3. agregar un `QLabel` .
4. cambiar su texto a `Hola mundo` .
5. aplicar un layout simple.
6. ejecutar la aplicación.



HelloWorld: qué observar

El ejemplo es pequeño, pero introduce el flujo completo:

- `mainwindow.ui` contiene la etiqueta y su texto.
- `ui->setupUi(this)` crea la interfaz al construir la ventana.
- `QLabel` es un widget; no requiere lógica adicional para mostrar texto estático.
- si se cambia el texto desde C++, se usa el `objectName` del label.

```
ui->lblMensaje->setText("Hola mundo");
```

El objetivo no es programar una aplicación compleja, sino reconocer dónde vive cada parte del proyecto.

Dibujo en Qt

Técnica	Uso típico
QPainter en paintEvent()	dibujo directo sobre un widget
QGraphicsView + QGraphicsScene	escena con varios objetos 2D

```
void Widget::paintEvent(QPaintEvent *)
{
    QPainter painter(this);
    painter.setPen(Qt::blue);
    painter.drawRect(10, 10, 120, 40);
}
```

Si necesita repintar, use `update()`.

QGraphicsView y QGraphicsScene

```
QGraphicsScene *scene = new QGraphicsScene(this);  
  
scene->addRect(10, 10, 100, 30,  
              QPen(Qt::blue),  
              QBrush(Qt::cyan));  
  
ui->graphicsView->setScene(scene);
```

`QGraphicsScene` administra items. `QGraphicsView` muestra una vista de la escena.

Es una buena base para simuladores gráficos con varios objetos visibles.

Animaciones con QPropertyAnimation

`QPropertyAnimation` cambia una propiedad Qt a lo largo del tiempo.

En JavaFX animábamos propiedades de nodos. En Qt la idea es la misma: cambiar propiedades visuales, como posición, opacidad o rotación.

```
QPropertyAnimation *anim = new QPropertyAnimation(objeto, "pos");  
  
anim->setDuration(2000);  
anim->setStartValue(QPointF(100, 100));  
anim->setEndValue(QPointF(400, 250));  
anim->setEasingCurve(QEasingCurve::InOutQuad);  
  
anim->start();
```

La propiedad debe existir en el sistema de metaobjetos de Qt.

Ejemplo: QPropertyAnimationCircle

En `examples/QPropertyAnimationCircle` :

- `AnimatedRect` hereda de `QGraphicsObject` , por eso puede animar `"pos"` .
- `boundingRect()` define el área del item; `paint()` solo dibuja.
- `QPropertyAnimation(rect, "pos", rect)` anima la posición y usa `rect` como padre.
- `setLoopCount(-1)` repite indefinidamente.
- `QEasingCurve::Linear` mantiene velocidad constante.

Los puntos del círculo se entregan con `setKeyValueAt(t, point)` . Qt interpola entre ellos y el `event loop` actualiza la escena.

Escena con objetos animados

Para un simulador gráfico genérico:

- `QGraphicsView` muestra la escena.
- `QGraphicsScene` contiene el fondo y los objetos.
- `QGraphicsObject` representa cada objeto animado.
- `QPropertyAnimation` mueve un item hacia otro estado.

```
void MovingItem::moveTo(const QPointF &dest)
{
    QPropertyAnimation *anim = new QPropertyAnimation(this, "pos");
    anim->setDuration(1500);
    anim->setStartValue(pos());
    anim->setEndValue(dest);
    anim->setEasingCurve(QEasingCurve::InOutQuad);
    anim->start(QAbstractAnimation::DeleteWhenStopped);
}
```

QPropertyAnimation o QTimer

Use `QPropertyAnimation` cuando el movimiento sea una transición entre estados.

```
item->moveTo(QPointF(400, 250));
```

Use `QTimer` cuando necesite una simulación continua paso a paso:

- física propia.
- colisiones.
- velocidad variable.
- actualización manual por frame.

QPropertyAnimation o QTimer

`QTimer` emite la señal `timeout()` cada cierto intervalo.

Esa señal se conecta a un slot donde se avanza el modelo y se actualiza la vista.

```
QTimer *timer = new QTimer(this);

connect(timer, &QTimer::timeout,
        this, &MainWindow::actualizarSimulacion);

timer->start(16); // cerca de 60 actualizaciones por segundo

void MainWindow::actualizarSimulacion()
{
    simulador.avanzarPaso();
    scene->update();
}
```

No use un ciclo `while` para animar: bloquearía el `event loop` y la interfaz dejaría de responder.

Recursos `.qrc`

Qt puede empaquetar recursos con `.qrc` :

- iconos, imágenes, QML, hojas de estilo o datos.
- acceso con rutas como `(":/icons/save.svg"` .
- inclusión desde CMake como fuentes o con `qt_add_resources` .

Ejemplo con una imagen:

```
<!-- resources.qrc -->
<RCC>
    <qresource prefix="/images">
        <file alias="logo.png">images/logo.png</file>
    </qresource>
</RCC>
```

La imagen queda disponible como `(":/images/logo.png"` .

Recursos `.qrc`: ejemplo

En Designer:

1. Agregar el archivo `.qrc` al proyecto y agrega una imagen a un prefix `/images/logo.png`.
2. Arrastrar un `QLabel` a la ventana.
3. En la propiedad `pixmap`, elegir el recurso `:/images/logo.png`.
4. Ajustar `scaledContents` si se quiere que la imagen se adapte al tamaño del label.

Flujo recomendado

1. Crear proyecto `Qt Widgets Application` con CMake.
2. Definir clases de dominio.
3. Diseñar la interfaz con layouts.
4. Nombrar widgets con `objectName` claros.
5. Conectar signals y slots en C++.
6. Probar interacciones pequeñas.
7. Mover cálculos fuera de `MainWindow`.

Errores frecuentes

Síntoma	Causa probable
<code>undefined reference to vtable</code>	falta <code>Q_OBJECT</code> , <code>moc</code> no corrió, o CMake no ve el archivo
No se ven cambios en la interfaz	falta <code>setupUi(this)</code> o se modifica otro objeto
Widgets se amontonan	faltan layouts
La aplicación se cierra	falta <code>app.exec()</code>
Doble liberación	mezcla incorrecta de padres Qt y objetos locales

Referencias

- Qt Widgets: <https://doc.qt.io/qt-6/qtwidgets-index.html>
- Signals & Slots: <https://doc.qt.io/qt-6/signalsandslots.html>
- QOverload: <https://doc.qt.io/qt-6/qoverload.html>
- Build with CMake: <https://doc.qt.io/qt-6/cmake-get-started.html>
- Object Trees & Ownership: <https://doc.qt.io/qt-6/objecttrees.html>
- Qt Quick: <https://doc.qt.io/qt-6/qtquick-index.html>
- Material TEL102: 06_QtLibrary

Anexo

Instalación de Qt 6

Proceso recomendado para Linux Debian/Ubuntu
usando Qt Online Installer.



Anexo: requisitos y entorno base

Antes de instalar:

- sistema Debian, Ubuntu o derivado actualizado.
- conexión a Internet.
- cuenta gratuita de Qt para acceder a la versión Open Source.
- herramientas de compilación C++ y CMake.

```
sudo apt update  
sudo apt install -y build-essential cmake
```

Si aparece el error `Could not load platform plugin "xcb"`, instale `libxcb-xinerama0`.

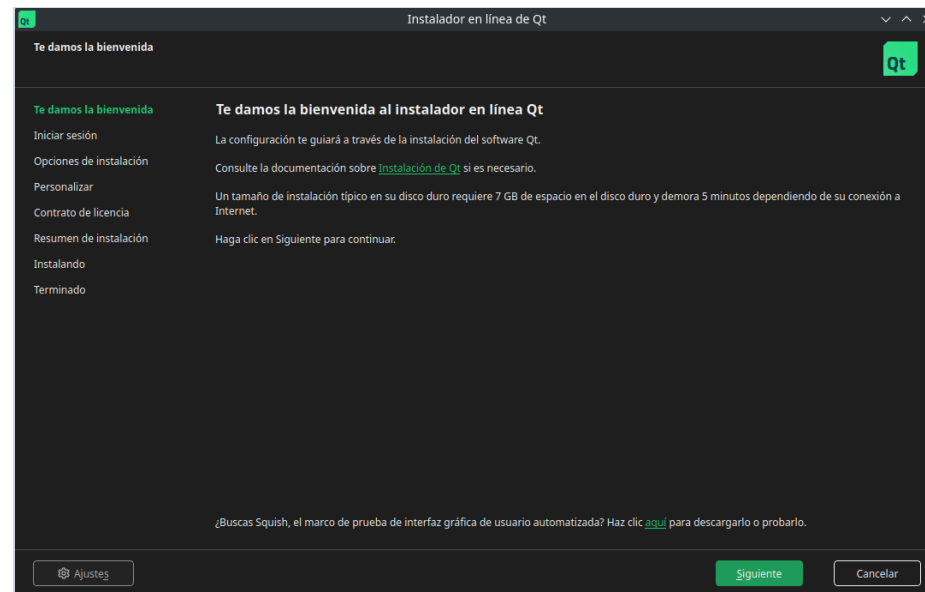
Anexo: descargar y ejecutar el instalador

Descargue el instalador desde:

<https://www.qt.io/download-open-source>

Archivo esperado: `qt-online-installer-linux-x64-<version>.run`

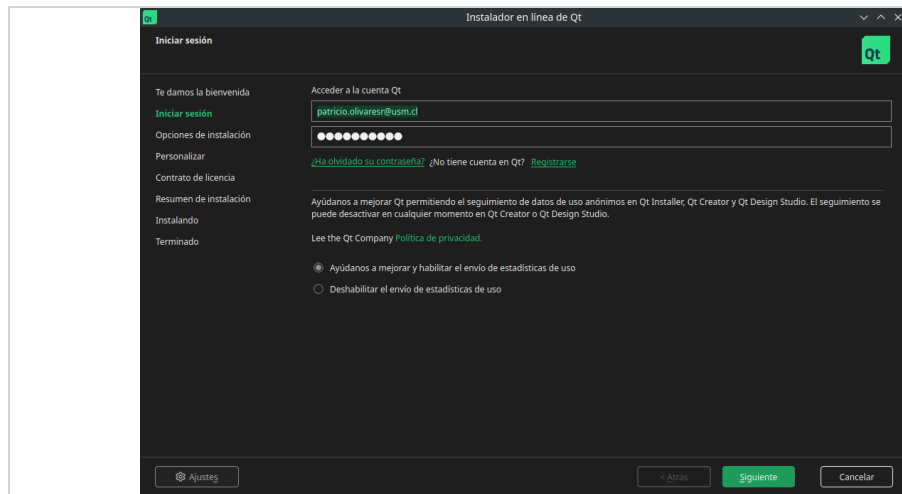
```
chmod +x qt-online-installer-linux-x64-<version>.run  
./qt-online-installer-linux-x64-<version>.run
```



Anexo: cuenta Qt y licencia

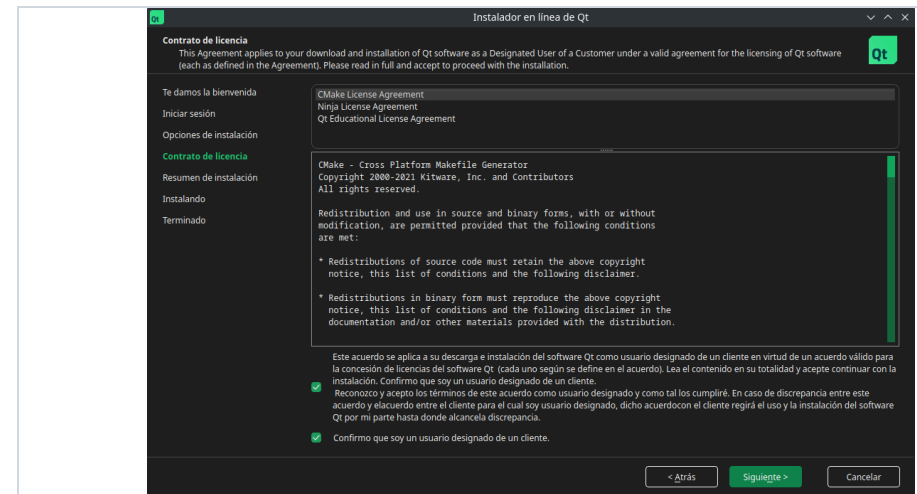
1. Iniciar sesión

Use una cuenta Qt gratuita o cree una cuenta nueva desde el instalador.



2. Aceptar licencia

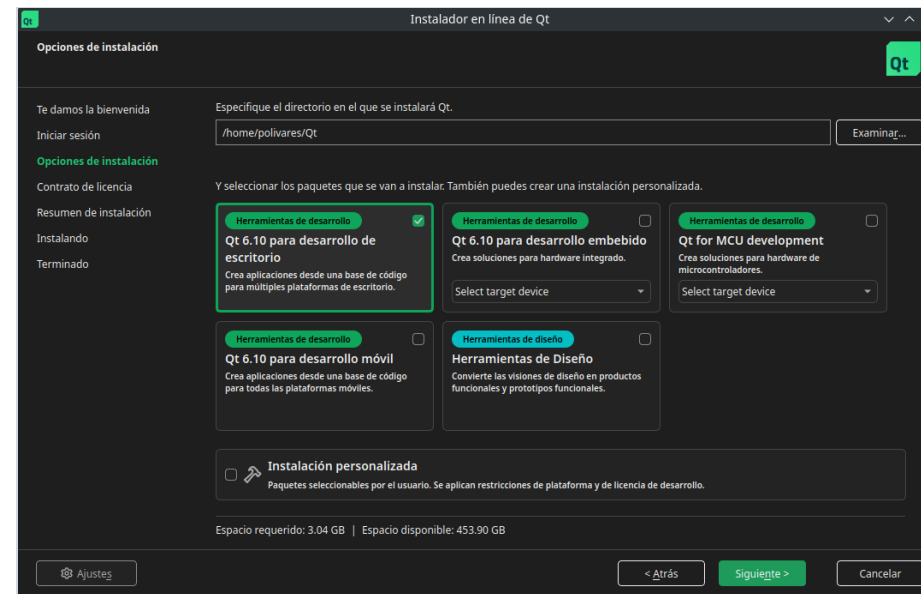
Acepte los términos de Qt y CMake para continuar con la instalación Open Source.



Anexo: seleccionar instalación de escritorio

Seleccione el directorio de instalación y el perfil:

- Qt 6 for Desktop Development .
- módulos principales de Qt 6.
- Qt Creator como entorno de desarrollo.



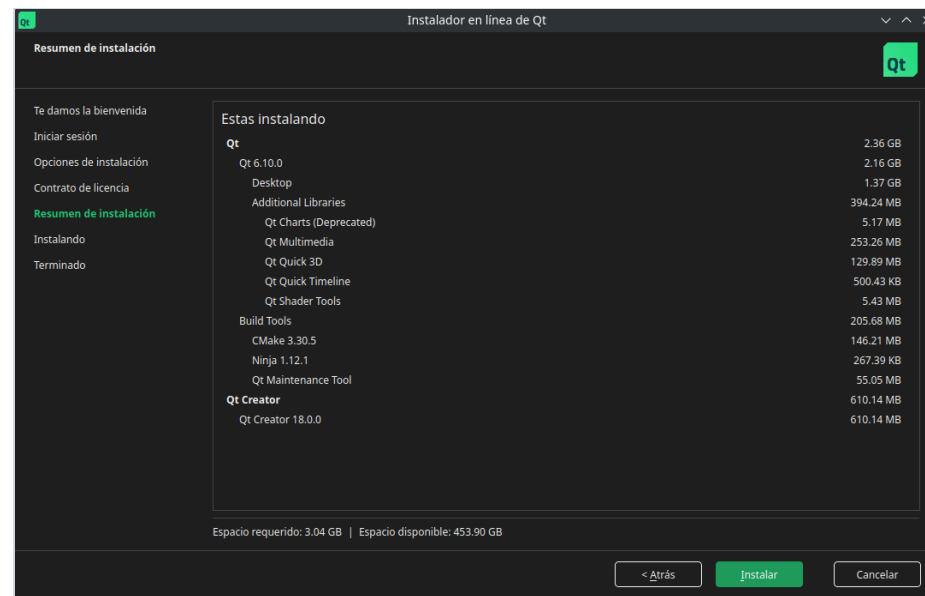
Anexo: revisar componentes

En el resumen revise que estén marcados:

- Qt 6 para desarrollo de escritorio.
- Qt Creator.
- Qt Tools.
- herramientas de compilación asociadas al kit de escritorio.

Para ELO329, el flujo recomendado es crear proyectos Qt Widgets con CMake desde Qt Creator.

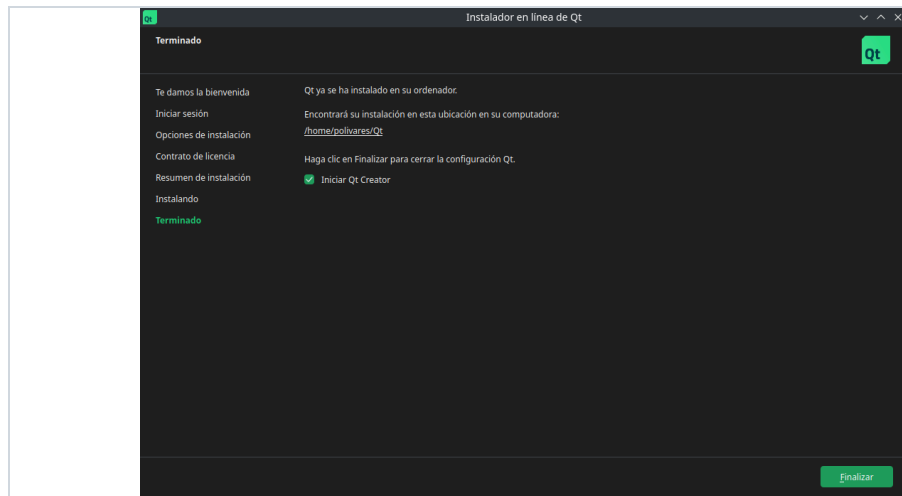
Anexo: revisar componentes



Anexo: finalizar y abrir Qt Creator

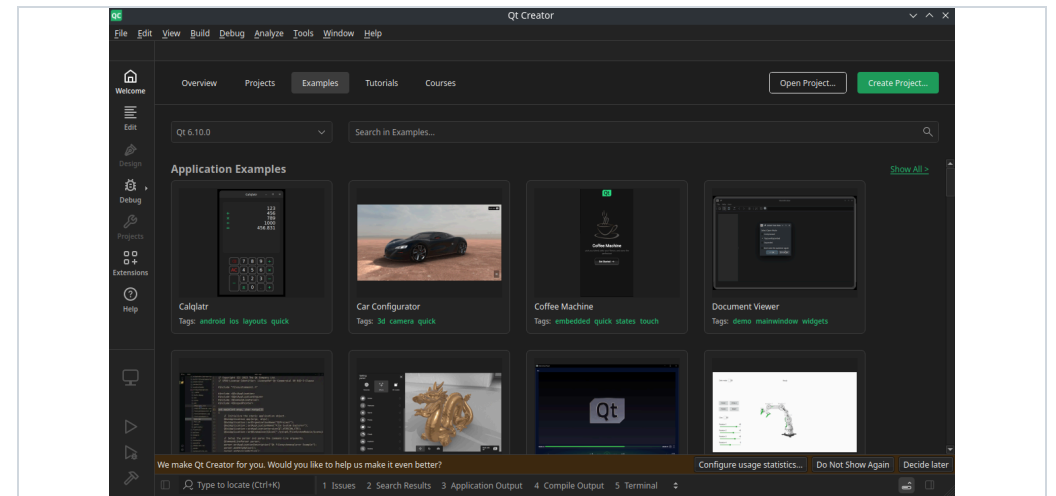
1. Terminar instalación

Espere a que finalice el proceso. Puede marcar la opción para iniciar Qt Creator al cerrar el instalador.



2. Crear proyecto

Qt Creator mostrará ejemplos, plantillas y los kits disponibles para comenzar un proyecto nuevo.



Anexo: verificación rápida

Compruebe versiones desde la terminal:

```
cmake --version  
qmake6 --version
```

Si ambos comandos muestran versiones recientes, el entorno está listo para compilar proyectos Qt 6 con CMake.

En Qt Creator, también revise que el proyecto use un kit de escritorio de Qt 6 antes de compilar.